

SQL Practice Exam

Window Functions — MySQL

Total Marks: 80 | Questions: 4 | Difficulty: Medium

Question 1: Days Between Customer Orders

Marks: 20

Database: CUSTOMERS_ORDERS_DB

Difficulty: Medium

Concept Tested: LAG() window function

1. Database Creation Script (MySQL)

```
-- Create database
CREATE DATABASE IF NOT EXISTS CUSTOMERS_ORDERS_DB;
USE CUSTOMERS_ORDERS_DB;

-- 1. Customers Table
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL
);

-- 2. Orders Table
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

-- Sample Data: Customers
INSERT INTO Customers (customer_id, customer_name) VALUES
(1, 'Aarav'),
(2, 'Diya'),
(3, 'Kabir');

-- Sample Data: Orders
INSERT INTO Orders (order_id, customer_id, order_date, amount) VALUES
(501, 1, '2025-08-02', 800.00),
(506, 1, '2025-08-07', 1200.00),
(514, 1, '2025-08-19', 950.00),
(602, 2, '2025-08-04', 650.00),
(609, 2, '2025-08-13', 1050.00),
(701, 3, '2025-08-06', 1500.00),
(708, 3, '2025-08-16', 1100.00);
```

2. Schema Details

Customers Table

Column Name	Data Type	Description
customer_id	INT (PK)	Unique customer ID

Column Name	Data Type	Description
customer_name	VARCHAR(100)	Customer name

Orders Table

Column Name	Data Type	Description
order_id	INT (PK)	Unique order ID
customer_id	INT (FK)	Customer who placed the order
order_date	DATE	Date when the order was placed
amount	DECIMAL(10,2)	Order amount

3. Problem Statement

For each order placed by a customer, calculate the number of days that have passed since the customer's immediately previous order.

Use the LAG() window function to retrieve the previous order date for the same customer.

Requirements

- Process the orders separately for each customer using customer_id.
- Use LAG() to retrieve the immediately previous order_date for each customer.
- Determine the sequence of orders using order_date and then order_id.
- Calculate the number of days between the current order date and the previous order date.
- For the first order of each customer, days_since_previous_order should be NULL.

Sort the final result by:

1. customer_name
2. order_date
3. order_id

Required Output Columns

- customer_name
- order_id
- order_date
- days_since_previous_order

Expected Output

customer_name	order_id	order_date	days_since_previous_order
Aarav	501	2025-08-02	NULL
Aarav	506	2025-08-07	5
Aarav	514	2025-08-19	12
Diya	602	2025-08-04	NULL

customer_name	order_id	order_date	days_since_previous_order
Diya	609	2025-08-13	9
Kabir	701	2025-08-06	NULL
Kabir	708	2025-08-16	10

Question 2: Event Average Finish Time

Marks: 20

Database: ATHLETICS_RESULTS_DB

Difficulty: Medium

Concept Tested: AVG() window function with PARTITION BY

1. Database Creation Script (MySQL)

```
-- Create database
CREATE DATABASE IF NOT EXISTS ATHLETICS_RESULTS_DB;
USE ATHLETICS_RESULTS_DB;

-- 1. Events Table
CREATE TABLE Events (
    event_id INT PRIMARY KEY,
    event_name VARCHAR(100) NOT NULL
);

-- 2. Athletes Table
CREATE TABLE Athletes (
    athlete_id INT PRIMARY KEY,
    athlete_name VARCHAR(100) NOT NULL
);

-- 3. Results Table
CREATE TABLE Results (
    result_id INT PRIMARY KEY,
    event_id INT NOT NULL,
    athlete_id INT NOT NULL,
    finish_time DECIMAL(5,2) NOT NULL,
    status ENUM('Recorded','Disqualified','DNS') NOT NULL,
    FOREIGN KEY (event_id) REFERENCES Events(event_id),
    FOREIGN KEY (athlete_id) REFERENCES Athletes(athlete_id)
);

-- Sample Data: Events
INSERT INTO Events (event_id, event_name) VALUES
(1, '100m Sprint'),
(2, '200m Sprint');
```

```
-- Sample Data: Athletes
INSERT INTO Athletes (athlete_id, athlete_name) VALUES
(1, 'Alice'),
(2, 'Brian'),
(3, 'Carlos'),
(4, 'Diana');

-- Sample Data: Results
INSERT INTO Results (result_id, event_id, athlete_id, finish_time, status) VALUES
(201, 1, 1, 11.20, 'Recorded'),
(202, 1, 2, 10.95, 'Recorded'),
(203, 1, 3, 12.10, 'Recorded'),
(204, 1, 4, 11.50, 'Disqualified'),
(205, 2, 1, 22.50, 'Recorded'),
(206, 2, 2, 21.80, 'Recorded'),
(207, 2, 3, 21.80, 'Recorded'),
(208, 2, 4, 23.10, 'DNS');
```

2. Schema Details

Events Table

Column Name	Data Type	Description
event_id	INT (PK)	Unique event ID
event_name	VARCHAR(100)	Name of the event

Athletes Table

Column Name	Data Type	Description
athlete_id	INT (PK)	Unique athlete ID
athlete_name	VARCHAR(100)	Name of the athlete

Results Table

Column Name	Data Type	Description
result_id	INT (PK)	Unique result ID
event_id	INT (FK)	Event in which the athlete participated
athlete_id	INT (FK)	Athlete who participated
finish_time	DECIMAL(5,2)	Finish time in seconds
status	ENUM('Recorded','Disqualified','DNS')	Status of the result

3. Problem Statement

A national athletics federation stores athlete performance data for multiple track events.

Generate a report that, for every recorded result, displays:

- The event name
- The athlete name
- The athlete's finish time in seconds
- The average finish time of all recorded performances within the same event

Use the AVG() window function with PARTITION BY to calculate the average finish time separately for each event.

Only results where status = 'Recorded' should be included.

Results with statuses such as Disqualified or DNS must not appear in the final output and must not be included while calculating the event average.

Round the calculated average finish time to 2 decimal places.

Sort the final result by:

1. event_name
2. finish_time
3. athlete_name

Required Output Columns

- event_name
- athlete_name
- finish_time
- avg_event_time

Expected Output

event_name	athlete_name	finish_time	avg_event_time
100m Sprint	Brian	10.95	11.42
100m Sprint	Alice	11.20	11.42
100m Sprint	Carlos	12.10	11.42
200m Sprint	Brian	21.80	22.03
200m Sprint	Carlos	21.80	22.03
200m Sprint	Alice	22.50	22.03

Question 3: Average Delivery Time by Route

Marks: 20

Database: DELIVERY_TRACKING_DB

Difficulty: Medium

Concept Tested: AVG() window function with PARTITION BY

1. Database Creation Script (MySQL)

```
-- Create database
```

```

CREATE DATABASE IF NOT EXISTS DELIVERY_TRACKING_DB;
USE DELIVERY_TRACKING_DB;

-- 1. Routes Table
CREATE TABLE Routes (
    route_id INT PRIMARY KEY,
    route_name VARCHAR(100) NOT NULL
);

-- 2. Drivers Table
CREATE TABLE Drivers (
    driver_id INT PRIMARY KEY,
    driver_name VARCHAR(100) NOT NULL
);

-- 3. Deliveries Table
CREATE TABLE Deliveries (
    delivery_id INT PRIMARY KEY,
    route_id INT NOT NULL,
    driver_id INT NOT NULL,
    delivery_time DECIMAL(6,2) NOT NULL,
    status ENUM('Completed','Delayed','Cancelled') NOT NULL,
    FOREIGN KEY (route_id) REFERENCES Routes(route_id),
    FOREIGN KEY (driver_id) REFERENCES Drivers(driver_id)
);

-- Sample Data: Routes
INSERT INTO Routes (route_id, route_name) VALUES
(1, 'Downtown Express'),
(2, 'Uptown Loop');

-- Sample Data: Drivers
INSERT INTO Drivers (driver_id, driver_name) VALUES
(1, 'Ravi'),
(2, 'Karan'),
(3, 'Farhan'),
(4, 'Aditya');

-- Sample Data: Deliveries
INSERT INTO Deliveries (delivery_id, route_id, driver_id, delivery_time, status)
VALUES
(301, 1, 1, 42.50, 'Completed'),
(302, 1, 2, 39.00, 'Completed'),
(303, 1, 3, 46.00, 'Completed'),
(304, 1, 4, 55.00, 'Delayed'),
(305, 2, 1, 61.00, 'Completed'),
(306, 2, 2, 58.50, 'Completed'),
(307, 2, 3, 63.50, 'Completed'),
(308, 2, 4, 70.00, 'Cancelled');

```

Note: the source question did not specify route_name / driver_name values, so reasonable sample values have been filled in above to make the script runnable.

2. Schema Details

Table	Column	Type	Description
Routes	route_id	INT (PK)	Unique route ID
Routes	route_name	VARCHAR(100)	Name of route
Drivers	driver_id	INT (PK)	Unique driver ID
Drivers	driver_name	VARCHAR(100)	Driver name
Deliveries	delivery_id	INT (PK)	Unique delivery ID
Deliveries	route_id	INT (FK)	Route used
Deliveries	driver_id	INT (FK)	Driver who delivered
Deliveries	delivery_time	DECIMAL(6,2)	Delivery time in minutes
Deliveries	status	ENUM	Completed, Delayed, Cancelled

3. Problem Statement

Generate a report for every completed delivery showing the route name, driver name, delivery time, and the average delivery time of all completed deliveries within the same route.

- Use AVG() as a window function with PARTITION BY route_id.
- Only status = 'Completed' should be included.
- Delayed and Cancelled deliveries must not appear and must not affect the route average.
- Round the calculated average delivery time to 2 decimal places.
- Sort by route_name, delivery_time, driver_name.

Required Output Columns

- route_name
- driver_name
- delivery_time
- avg_route_time

Sample Data

delivery_id	route_id	driver_id	delivery_time	status
301	1	1	42.50	Completed
302	1	2	39.00	Completed
303	1	3	46.00	Completed
304	1	4	55.00	Delayed
305	2	1	61.00	Completed
306	2	2	58.50	Completed
307	2	3	63.50	Completed
308	2	4	70.00	Cancelled

Expected Output

route_name	driver_name	delivery_time	avg_route_time
Downtown Express	Karan	39.00	42.50
Downtown Express	Ravi	42.50	42.50
Downtown Express	Farhan	46.00	42.50
Uptown Loop	Karan	58.50	61.00
Uptown Loop	Ravi	61.00	61.00
Uptown Loop	Farhan	63.50	61.00

Question 4: Days Between Patient Appointments

Marks: 20

Database: PATIENT_APPOINTMENTS_DB

Difficulty: Medium

Concept Tested: LAG() window function

1. Database Creation Script (MySQL)

```
-- Create database
CREATE DATABASE IF NOT EXISTS PATIENT_APPOINTMENTS_DB;
USE PATIENT_APPOINTMENTS_DB;

-- 1. Patients Table
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    patient_name VARCHAR(100) NOT NULL
);

-- 2. Appointments Table
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    patient_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    fee DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);

-- Sample Data: Patients
INSERT INTO Patients (patient_id, patient_name) VALUES
(1, 'Meera'),
(2, 'Rohan'),
(3, 'Sahil');

-- Sample Data: Appointments
```

```

INSERT INTO Appointments (appointment_id, patient_id, appointment_date, fee)
VALUES
(801, 1, '2026-08-02', 700.00),
(806, 1, '2026-08-08', 850.00),
(814, 1, '2026-08-21', 700.00),
(902, 2, '2026-08-04', 600.00),
(909, 2, '2026-08-18', 750.00),
(1001, 3, '2026-08-06', 900.00),
(1008, 3, '2026-08-16', 900.00);

```

2. Schema Details

Table	Column	Type	Description
Patients	patient_id	INT (PK)	Unique patient ID
Patients	patient_name	VARCHAR(100)	Patient name
Appointments	appointment_id	INT (PK)	Unique appointment ID
Appointments	patient_id	INT (FK)	Patient attending
Appointments	appointment_date	DATE	Appointment date
Appointments	fee	DECIMAL(10,2)	Appointment fee

3. Problem Statement

For each appointment, calculate the number of days that have passed since the patient's immediately previous appointment.

- Process appointments separately for each patient using patient_id.
- Use LAG() to retrieve the immediately previous appointment_date for each patient.
- Determine appointment sequence using appointment_date and then appointment_id.
- Calculate the number of days between the current and previous appointment.
- For the first appointment of each patient, days_since_previous_appointment should be NULL.
- Sort by patient_name, appointment_date, appointment_id.

Required Output Columns

- patient_name
- appointment_id
- appointment_date
- days_since_previous_appointment

Sample Data

Patients

patient_id	patient_name
1	Meera
2	Rohan

patient_id	patient_name
3	Sahil

Appointments

appointment_id	patient_id	appointment_date	fee
801	1	2026-08-02	700.00
806	1	2026-08-08	850.00
814	1	2026-08-21	700.00
902	2	2026-08-04	600.00
909	2	2026-08-18	750.00
1001	3	2026-08-06	900.00
1008	3	2026-08-16	900.00

Expected Output

patient_name	appointment_id	appointment_date	days_since_previous_appointment
Meera	801	2026-08-02	NULL
Meera	806	2026-08-08	6
Meera	814	2026-08-21	13
Rohan	902	2026-08-04	NULL
Rohan	909	2026-08-18	14
Sahil	1001	2026-08-06	NULL
Sahil	1008	2026-08-16	10

Solutions

Solution — Question 1: Days Between Customer Orders

```
SELECT
    c.customer_name,
    o.order_id,
    o.order_date,
    DATEDIFF(
        o.order_date,
        LAG(o.order_date) OVER (
            PARTITION BY o.customer_id
            ORDER BY o.order_date, o.order_id
        )
    ) AS days_since_previous_order
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
ORDER BY
    c.customer_name,
    o.order_date,
    o.order_id;
```

Explanation: The LAG() function, partitioned by customer_id and ordered by order_date then order_id, returns the order_date of the immediately preceding order for that same customer. DATEDIFF() then computes the gap in days between the current order_date and that previous date. For a customer's very first order, LAG() returns NULL, so DATEDIFF() correctly evaluates to NULL.

Solution — Question 2: Event Average Finish Time

```
SELECT
    event_name,
    athlete_name,
    finish_time,
    avg_event_time
FROM (
    SELECT
        e.event_name,
        a.athlete_name,
        r.finish_time,
        ROUND(
            AVG(r.finish_time) OVER (PARTITION BY r.event_id),
            2
        ) AS avg_event_time
    FROM Results r
    JOIN Events e ON e.event_id = r.event_id
    JOIN Athletes a ON a.athlete_id = r.athlete_id
    WHERE r.status = 'Recorded'
) AS ranked
ORDER BY
    event_name,
    finish_time,
    athlete_name;
```

Explanation: The WHERE clause filters out Disqualified and DNS results before the window function runs, so only Recorded rows are used both in the output and in the average calculation. AVG(finish_time) OVER (PARTITION BY event_id) computes the average finish time per event without collapsing rows (unlike GROUP BY), so each individual athlete's finish_time is still visible alongside the event average. The outer query and ORDER BY then produce the final sorted report.

Solution — Question 3: Average Delivery Time by Route

```
SELECT
    route_name,
    driver_name,
    delivery_time,
    avg_route_time
FROM (
    SELECT
        rt.route_name,
        dr.driver_name,
        d.delivery_time,
        ROUND(
            AVG(d.delivery_time) OVER (PARTITION BY d.route_id),
            2
        ) AS avg_route_time
    FROM Deliveries d
    JOIN Routes rt    ON rt.route_id = d.route_id
    JOIN Drivers dr   ON dr.driver_id = d.driver_id
    WHERE d.status = 'Completed'
) AS ranked
ORDER BY
    route_name,
    delivery_time,
    driver_name;
```

Explanation: The WHERE clause keeps only Completed deliveries before the window function runs, so Delayed and Cancelled rows never enter the output or the average. AVG(delivery_time) OVER (PARTITION BY route_id) computes each route's average delivery time while keeping every individual delivery row visible, unlike GROUP BY which would collapse them. The result is then sorted by route_name, delivery_time, and driver_name as required.

Solution — Question 4: Days Between Patient Appointments

```
SELECT
    p.patient_name,
    a.appointment_id,
    a.appointment_date,
    DATEDIFF(
        a.appointment_date,
        LAG(a.appointment_date) OVER (
            PARTITION BY a.patient_id
            ORDER BY a.appointment_date, a.appointment_id
        )
    ) AS days_since_previous_appointment
FROM Appointments a
JOIN Patients p ON p.patient_id = a.patient_id
```

```
ORDER BY
  p.patient_name,
  a.appointment_date,
  a.appointment_id;
```

Explanation: LAG(), partitioned by patient_id and ordered by appointment_date then appointment_id, retrieves the previous appointment_date for the same patient. DATEDIFF() subtracts that previous date from the current appointment_date to get the day gap. For a patient's first appointment, LAG() returns NULL, so days_since_previous_appointment correctly evaluates to NULL.